

WPF + C#

СОЗДАЁМ СВОЙ БРАУЗЕР

Пошаговая памятка

← → ↺ 🏠	https://ya.ru	ПЕРЕЙТИ
Здесь будет открываться настоящий сайт		
Цель: за одно занятие собрать собственное Windows-приложение		

Что получится к концу занятия

Наш браузер научится:

- открывать сайты по адресу;
- переходить назад и вперёд;
- обновлять страницу;
- возвращаться на домашний сайт;
- показывать в адресной строке адрес открытой страницы.



Можно сделать по-своему

Название окна, цвет кнопок, символы на кнопках, размер окна, домашний сайт и подпись «Перейти» можно менять. Так у каждого получится свой браузер.

Что уже подготовлено преподавателем

- Создан проект WPF на современном .NET.
- Через NuGet установлен пакет Microsoft.Web.WebView2.
- Проект запускается без ошибок.

На старте в проекте ещё нет самого элемента WebView2 и нет строки, которая автоматически открывает сайт. Их добавим сами по ходу работы.

Главная идея

XAML	C#	WebView2
Интерфейс: кнопки, поле адреса, размеры	Логика: что делать при нажатии	Готовый компонент для показа сайтов

Шаг 1. Настраиваем окно

Открой файл MainWindow.xaml. В самом верху находится описание окна.

```
<Window x:Class="WpfApp2.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Мой браузер"
        Height="700"
        Width="1100">
```

Что здесь важно

Title	текст в заголовке окна
Height	высота окна
Width	ширина окна
x:Class	связывает XAML с C# — эту строку не меняем
 Попробуй настроить Измени Title="Мой браузер" на своё название: например «Браузер Максима», «Green Browser» или «Школьный браузер». Height и Width тоже можно менять.	

Шаг 2. Делим окно на две части

Нам нужны две области: сверху — панель управления, ниже — сайт. Для этого используем Grid.

```
<Grid Background="#F3F4F6">

    <Grid.RowDefinitions>
        <RowDefinition Height="65"/>
        <RowDefinition Height="*"/>
    </Grid.RowDefinitions>

</Grid>
```

Height="65" — верхняя строка высотой 65 пикселей. **Height="*"** — всё оставшееся место.

Можно изменить

Background="#F3F4F6" — цвет фона окна. Попробуй, например, #FFFFFF, #E0F2FE или #FEF3C7.

Шаг 3. Готовим места для кнопок

В верхней строке создаём ещё один Grid. Он будет нашей панелью инструментов.

```
<Grid Grid.Row="0" Margin="10">

    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="55"/>
        <ColumnDefinition Width="55"/>
        <ColumnDefinition Width="55"/>
        <ColumnDefinition Width="55"/>
        <ColumnDefinition Width="*"/>
        <ColumnDefinition Width="130"/>
    </Grid.ColumnDefinitions>

</Grid>
```

Первые четыре колонки — кнопки. Пятая растягивается и станет адресной строкой. Последняя — кнопка «Перейти».

Не страшно

Grid можно представить как таблицу без видимых границ. Grid.Column="0" означает «положить элемент в первую колонку». Нумерация в программировании часто начинается с нуля.

Шаг 4. Добавляем кнопки навигации

Внутри верхнего Grid добавь четыре кнопки:

```
<Button Grid.Column="0"
        Content="←"
        FontSize="20"
        Background="#22C55E"
        Foreground="White"
        Click="BackButton_Click"/>

<Button Grid.Column="1"
        Content="→"
        FontSize="20"
        Background="#22C55E"
        Foreground="White"
        Click="ForwardButton_Click"/>

<Button Grid.Column="2"
        Content="↻"
        FontSize="20"
        Background="#22C55E"
        Foreground="White"
        Click="RefreshButton_Click"/>

<Button Grid.Column="3"
        Content="⌂"
        FontSize="20"
        Background="#22C55E"
        Foreground="White"
        Click="HomeButton_Click"/>
```

Как читать одну кнопку

Grid.Column="0"	первая колонка
Content="←"	что написано на кнопке
FontSize="20"	размер символа
Background="#22C55E"	цвет кнопки
Click="BackButton_Click"	какой C#-код запустится при клике

Сделай свой дизайн

Можно безопасно менять Content, FontSize, Background и Foreground. Например, заменить зелёный #22C55E на синий #2563EB или фиолетовый #7C3AED.

Шаг 5. Добавляем адресную строку и кнопку «Перейти»

```
<TextBox Grid.Column="4"
        x:Name="AddressBox"
        FontSize="16"
        Margin="8,2"
        Padding="10"/>

<Button Grid.Column="5"
        Content="Перейти"
        FontSize="16"
        Background="#22C55E"
        Foreground="White"
        Click="GoButton_Click"/>
```

x:Name="AddressBox" — мы дали полное имя. Благодаря этому C# сможет обратиться к нему и прочитать введённый адрес.

✂ Можно изменить

Content="Перейти" можно заменить на «Открыть», «GO», «Вперёд!» или другой текст. Но **x:Name="AddressBox"** пока лучше оставить без изменений.

Шаг 6. Добавляем настоящий браузер — WebView2

Теперь впервые подключаем область, в которой будут открываться сайты.

Сначала добавь в начало тега Window ещё одну строку пространства имён:

```
xmlns:mv2="clr-namespace:Microsoft.Web.WebView2.Wpf;assembly=Microsoft.Web.WebView2.Wpf"
```

Затем после верхней панели, но внутри основного Grid, добавь:

```
<mv2:WebView2 Grid.Row="1"
        x:Name="Browser"
        Margin="10"/>
```

x:Name="Browser" — имя нашего браузера. Именно по этому имени мы будем обращаться к нему из C#.

⚠ Важно

Строку **xmlns:mv2** лучше копировать внимательно. Даже одна пропущенная буква может вызвать несколько непонятных ошибок.

Шаг 7. Открываем первый сайт

Перейди в **MainWindow.xaml.cs**. В конструкторе после **InitializeComponent()**; добавим первую команду браузеру.

```
public MainWindow()
{
    InitializeComponent();

    Browser.Source = new Uri("https://ya.ru");
}
```

Читаем по смыслу: у объекта **Browser** устанавливаем **Source** — адрес страницы. **new Uri(...)** превращает текст в интернет-адрес.

🏠 Выбери домашний сайт

Вместо `https://ya.ru` можно поставить любой адрес из списка в конце памятки. Не забудь оставить кавычки.

Шаг 8. Оживляем кнопку «Перейти»

Добавь метод в класс `MainWindow`:

```
private void GoButton_Click(object sender, RoutedEventArgs e)
{
    string address = AddressBox.Text;

    if (!address.StartsWith("http"))
    {
        address = "https://" + address;
    }

    Browser.Source = new Uri(address);
}
```

Разбираем по строкам

`string address = AddressBox.Text;`

берём текст из адресной строки и сохраняем в переменную `address`

`if (!address.StartsWith("http"))`

если адрес не начинается с `http...`

`address = "https://" + address;`

...добавляем `https://` автоматически

`Browser.Source = new Uri(address);`

открываем получившийся адрес

Шаг 9. Кнопки «Назад», «Вперёд», «Обновить», «Домой»

Теперь добавим короткие обработчики. Их можно копировать, но попробуй прочитать каждую строку.

Назад

```
private void BackButton_Click(object sender, RoutedEventArgs e)
{
    if (Browser.CanGoBack)
    {
        Browser.GoBack();
    }
}
```

Если браузер может перейти назад — переходим назад.

Вперёд

```
private void ForwardButton_Click(object sender, RoutedEventArgs e)
{
    if (Browser.CanGoForward)
    {
        Browser.GoForward();
    }
}
```

Логика почти такая же. Попробуй заметить, какие слова изменились.

Обновить

```
private void RefreshButton_Click(object sender, RoutedEventArgs e)
{
    Browser.Reload();
}
```

Домой

```
private void HomeButton_Click(object sender, RoutedEventArgs e)
{
    Browser.Source = new Uri("https://ya.ru");
}
```

Персонализация

Адрес домашней страницы можно заменить на другой. Главное — чтобы адрес начинался с `https://` и был внутри кавычек.

Шаг 10. Адресная строка должна следить за страницей

Если мы нажмём ссылку внутри сайта, `WebView2` перейдёт на новую страницу. Сделаем так, чтобы `AddressBox` автоматически показывал новый адрес.

В XAML дпиши событие `SourceChanged`:

```
<wv2:WebView2 Grid.Row="1"
    x:Name="Browser"
    Margin="10"
    SourceChanged="Browser_SourceChanged"/>
```

В `MainWindow.xaml.cs` сверху добавь:

```
using Microsoft.Web.WebView2.Core;
```

И добавь метод:

```
private void Browser_SourceChanged(
    object sender,
    CoreWebView2SourceChangedEventArgs e)
{
    AddressBox.Text = Browser.Source.ToString();
}
```

Главная строка здесь: `AddressBox.Text = Browser.Source.ToString();` — берём текущий адрес браузера и показываем его в текстовом поле.

Не обязательно запоминать

`CoreWebView2SourceChangedEventArgs` выглядит сложно — и это нормально. Сейчас достаточно понимать идею события: «адрес страницы изменился → выполни этот код».

Итоговый C#-код

Если где-то запутался, сравни свой файл MainWindow.xaml.cs с этой версией.

```
using System;
using System.Windows;
using Microsoft.Web.WebView2.Core;

namespace WpfApp2
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();

            Browser.Source = new Uri("https://ya.ru");
        }

        private void GoButton_Click(object sender, RoutedEventArgs e)
        {
            string address = AddressBox.Text;

            if (!address.StartsWith("http"))
            {
                address = "https://" + address;
            }

            Browser.Source = new Uri(address);
        }

        private void BackButton_Click(object sender, RoutedEventArgs e)
        {
            if (Browser.CanGoBack)
            {
                Browser.GoBack();
            }
        }

        private void ForwardButton_Click(object sender, RoutedEventArgs e)
        {
            if (Browser.CanGoForward)
            {
                Browser.GoForward();
            }
        }

        private void RefreshButton_Click(object sender, RoutedEventArgs e)
        {
            Browser.Reload();
        }

        private void HomeButton_Click(object sender, RoutedEventArgs e)
        {
            Browser.Source = new Uri("https://ya.ru");
        }

        private void Browser_SourceChanged(
            object sender,
            CoreWebView2SourceChangedEventArgs e)
        {
        }
    }
}
```

```

        AddressBox.Text = Browser.Source.ToString();
    }
}

```

Что можно менять и не бояться

Что менять	Пример	Что получится
Title	Title="Green Browser"	другое название окна
Background	Background="#2563EB"	другой цвет кнопок/фона
Content	Content="Открыть"	другая надпись на кнопке
FontSize	FontSize="18"	крупнее или мельче текст
Height / Width	Width="1200"	размер окна
Домашний адрес	"https://mail.ru"	другая стартовая страница

Что пока лучше не переименовывать

- x:Name="Browser"
- x:Name="AddressBox"
- Click="GoButton_Click" и другие имена обработчиков
- x:Class="WpfApp2.MainWindow"

Эти имена связывают XAML и C#. Переименовать их можно, но тогда нужно поменять такое же имя во всех связанных местах.

Бонус, если успел раньше остальных

Переход по Enter

Чтобы сайт открывался не только по кнопке, но и по клавише Enter, в TextBox добавь:

```
KeyDown="AddressBox_KeyDown"
```

А в C#:

```

private void AddressBox_KeyDown(
    object sender,
    System.Windows.Input.KeyEventArgs e)
{
    if (e.Key == System.Windows.Input.Key.Enter)
    {
        GoButton_Click(sender, e);
    }
}

```

Мини-челлендж

Сделай браузер узнаваемым: придумай название, выбери цвет, домашнюю страницу и подписи кнопок. Затем покажи соседу, чем твоя версия отличается.

Адреса сайтов для проверки браузера

Для занятия лучше использовать крупные российские сервисы. На дату подготовки памятки (29.09.2026) перечисленные ниже сайты доступны в России. Доступность интернет-ресурсов со временем может меняться.

Сайт	Адрес	Для чего попробовать
Яндекс	https://ya.ru	поиск
Mail.ru	https://mail.ru	портал и почта
RUTUBE	https://rutube.ru	видео
Кинопоиск	https://www.kinopoisk.ru	фильмы и сериалы
Яндекс Игры	https://yandex.ru/games/	браузерные игры
Яндекс Карты	https://yandex.ru/maps/	карты и поиск мест
Stepik	https://stepik.org	обучение
Habr	https://habr.com	статьи об IT
VK Видео	https://vkvideo.ru	видео
Яндекс Погода	https://yandex.ru/pogoda/	погода
Авито	https://www.avito.ru	объявления
☑ Совет Для первого теста лучше использовать https://ya.ru : короткий адрес легче набрать, и сразу видно, что WebView2 действительно работает.		

Если что-то не работает

Проблема	Что проверить
InitializeComponent не существует	Проверь, совпадают ли x:Class в XAML и namespace в C#. Затем пересобери проект.
WebView2 подчёркнут красным	Проверь установку Microsoft.Web.WebView2 и строку xmlns:ww2.
Кнопка нажимается, но ничего не происходит	Проверь имя Click в XAML и имя метода в C# — они должны совпадать.
C# не видит AddressBox или Browser	Проверь x:Name у элемента в XAML.
Адрес не открывается	Попробуй полный адрес с https:// и проверь, нет ли пробелов.